

VITE & GOURMAND

Documentation de gestion de projet

Organisation, suivi, Git, qualité et bilan

Projet ECF - TP Développeur Web et Web Mobile

Kelly FURNARI

Objet du document

Présenter la manière dont le projet Vite & Gourmand a été organisé, suivi, développé, testé, déployé et finalisé. Le document complète l'historique du dépôt Git public et répond au livrable de gestion de projet demandé dans l'ECF.

Synthèse

Vite & Gourmand est une application web de traiteur permettant de consulter des menus, créer un compte, commander une prestation et suivre son traitement. L'application comporte également des espaces dédiés aux employés et à l'administrateur.

Organisation retenue. Le projet a été conduit de manière itérative, avec un suivi léger adapté à un développement individuel. GitHub constitue le centre de traçabilité : dépôt public, onglet Issues, branches, commits et fusions. Les tâches ont été regroupées par domaines fonctionnels, puis réalisées dans des branches dédiées.

Flux de livraison. Les fonctionnalités partent de develop, sont développées dans une branche feature/*, refactor/* ou docs/*, puis sont testées avant fusion dans develop. Après validation globale, develop est fusionnée dans main, qui représente la version livrable.

Qualité. La validation repose sur les tests automatisés, les builds de production, les contrôles Git, les tests de déploiement et une recette manuelle des principaux parcours utilisateurs.

Élément	Choix appliqué
Outil de gestion	GitHub Issues, complété par l'historique Git
Méthode	Itérative, proche d'un Kanban léger
Branches	main, develop, feature/*, refactor/*, docs/*
Gestion des versions	Commits courts et explicites : feat, fix, refactor, docs, merge, release
Validation	Tests, builds, vérifications de déploiement et recette manuelle
Publication	Render pour le front-end et le back-end, Neon, Atlas et Brevo

Liens du projet

Dépôt public : <https://github.com/Scodyx/Vite-et-gourmand>

Gestion de projet : <https://github.com/Scodyx/Vite-et-gourmand/issues>

Application : <https://vite-gourmand-ztdw.onrender.com>

API : <https://vite-gourmand-api-5a1o.onrender.com>

1 - Contexte et méthode de travail

1.1 Contexte du projet

Le projet répond à un cahier des charges complet : interface publique, authentification, catalogue de menus, commandes, suivi des statuts, avis, gestion employé, administration, statistiques SQL/NoSQL, envoi d'e-mails et déploiement. Cette diversité impose de découper le travail afin de limiter les risques et de tester chaque domaine indépendamment.

1.2 Méthode retenue

Une méthode itérative proche d'un Kanban léger a été utilisée. Elle est adaptée à un projet individuel : le travail est découpé en tâches courtes, priorisées selon les dépendances techniques et la valeur métier. Chaque tâche produit un résultat vérifiable avant le passage à la suivante.

- Prioriser d'abord les fondations : données, sécurité, API et structure Angular.
- Développer ensuite les parcours métier de bout en bout.
- Tester avant chaque fusion importante.
- Déployer tôt afin de détecter les contraintes d'hébergement.
- Finaliser par la qualité visuelle, la lisibilité du code et les livrables.

Adaptation au contexte individuel. L'outil de suivi principal est GitHub Issues, mais la traçabilité repose également sur les branches et les commits. Dans une équipe plus large, chaque tâche aurait été systématiquement associée à un responsable, une estimation, un jalon et une revue de code formelle.

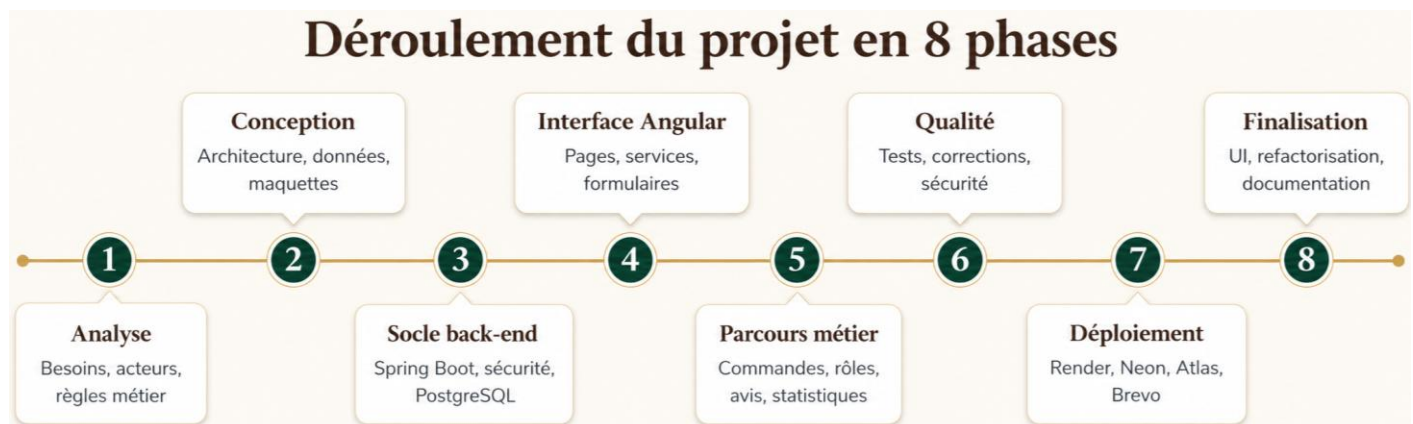


Figure 1 - Phases principales du projet

2 - Découpage du besoin et suivi des tâches

2.1 Regroupement fonctionnel

Le cahier des charges a été transformé en lots fonctionnels. Ce découpage a permis de traiter les dépendances dans le bon ordre et d'éviter de mélanger plusieurs objectifs dans une même branche.

Lot	Contenu principal	Critère de fin
Socle technique	Structure Spring Boot/Angular, bases SQL et NoSQL, configuration locale	Applications démarrables localement
Authentification	Inscription, connexion, JWT, renouvellement, mot de passe oublié	Accès sécurisé selon le rôle
Catalogue	Menus, plats, allergènes, horaires, filtres	Consultation publique et gestion métier
Commandes	Création, tarification, livraison, annulation, historique	Parcours client complet
Employé	Liste, filtres, détail et changement de statut	Traitement opérationnel des commandes
Administration	Employés, catalogue, horaires, avis et statistiques	Pilotage complet de l'application

2.2 Structure recommandée d'un ticket

Modèle de ticket

Titre : [Domaine] Résultat attendu

Contexte : pourquoi la tâche est nécessaire

Critères d'acceptation : conditions vérifiables

Sous-tâches : frontend, backend, données, tests

Validation : commandes à exécuter et parcours à vérifier

Résultat : branche et commit associés

2.3 Priorisation

- Priorité 1 - Bloquant : démarrage, base de données, sécurité, déploiement.
- Priorité 2 - Fonctionnel : parcours exigés dans le cahier des charges.
- Priorité 3 - Qualité : tests, ergonomie, accessibilité et lisibilité.
- Priorité 4 - Finalisation : documentation, maquettes et préparation du jury.

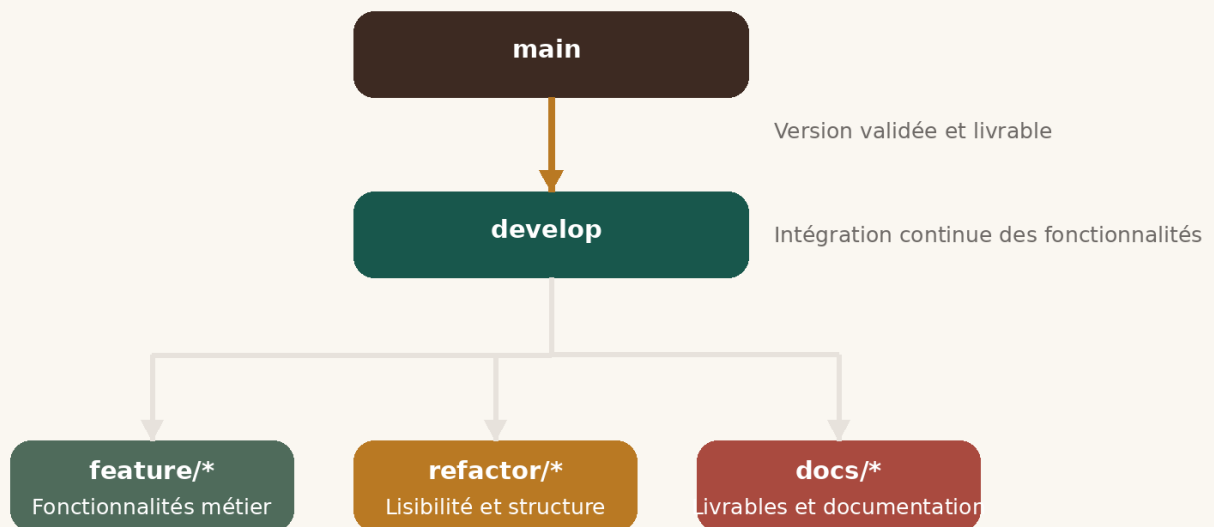
Amélioration identifiée. Pour un prochain projet, tous les besoins seraient saisis dans GitHub Issues dès le début, avec des étiquettes, des jalons et un tableau Kanban. Cela rendrait la progression plus visible sans dépendre de la lecture de l'historique Git.

3 - Stratégie Git et gestion des versions

3.1 Rôle des branches

Branche	Rôle	Règle
main	Version stable et livrable	Reçoit develop après validation complète
develop	Branche d'intégration	Réunit les fonctionnalités testées
feature/*	Développement fonctionnel	Une branche par fonctionnalité ou correctif métier
refactor/*	Amélioration sans changement fonctionnel	Lisibilité, structure ou organisation du code
docs/*	Documents et livrables	PDF, README et documentation

Organisation des branches Git



Principe : création depuis develop → développement et tests → fusion dans develop → validation globale → fusion dans main.

Figure 2 - Flux de branches appliqué

3.2 Conventions de commit

Préfixe	Utilisation	Exemple
feat	Ajout d'une fonctionnalité	feat: add order status workflow
fix	Correction d'un défaut	fix: validate minimum guest count
refactor	Amélioration interne	refactor: improve backend Java readability
docs	Documentation et livrables	docs: add graphic charter PDF
merge	Fusion explicite dans develop	merge: add final user manual
release	Publication finale	release: complete Vite & Gourmand application

3.3 Exemples issus du projet

Étape	Référence	Résultat
Préparation production	e4e9c6a puis merge 899ffa1	Docker, configuration production et Render
Modernisation de l'interface	cf79bf9 puis merge 5503b4f	Amélioration visuelle générale
Navigation	db0871e puis merge 75ab216	Navigation et tableaux de bord affinés
Manuel utilisateur	1a50e05	Ajout du PDF final
Charte graphique	47ceb87	Ajout du PDF et des six maquettes
Publication de la charte	8f45045 sur main	Livrable disponible sur la branche principale

Principe de sécurité

Les mots de passe, clés SMTP, secrets JWT et URI complètes de bases de données ne sont jamais enregistrés dans Git. Seules des valeurs d'exemple figurent dans .env.example et le README.

4 - Cycle de réalisation d'une tâche



Figure 3 - Cycle de vie d'une tâche

1. Analyse du besoin - Lecture du cahier des charges, identification des acteurs, des données, des règles métier et des cas d'erreur.

2. Définition du résultat - Formulation de critères d'acceptation vérifiables : comportement attendu, rôle autorisé, données retournées, message d'erreur et tests.

3. Création de branche - Création depuis develop avec un nom explicite : feature/..., refactor/... ou docs/....

4. Implémentation progressive - Modifications limitées au périmètre de la tâche, commits ciblés et vérification régulière de git status.

5. Validation locale - Tests automatisés, build, contrôle des fichiers modifiés et recette manuelle du parcours concerné.

6. Fusion dans develop - Fusion avec --no-ff afin de conserver une étape identifiable dans l'historique.

7. Validation globale - Vérification de l'application intégrée et, pour les changements déployables, contrôle du service Render.

8. Publication dans main - Fusion de develop dans main lorsque la version est considérée comme livrable.

Définition de terminé

Une tâche est considérée comme terminée lorsque le comportement attendu fonctionne, les tests passent, le build est valide, aucun secret n'est exposé, la branche est fusionnée et la documentation est mise à jour si nécessaire.

5 - Qualité, tests et validation

Contrôles qualité réalisés

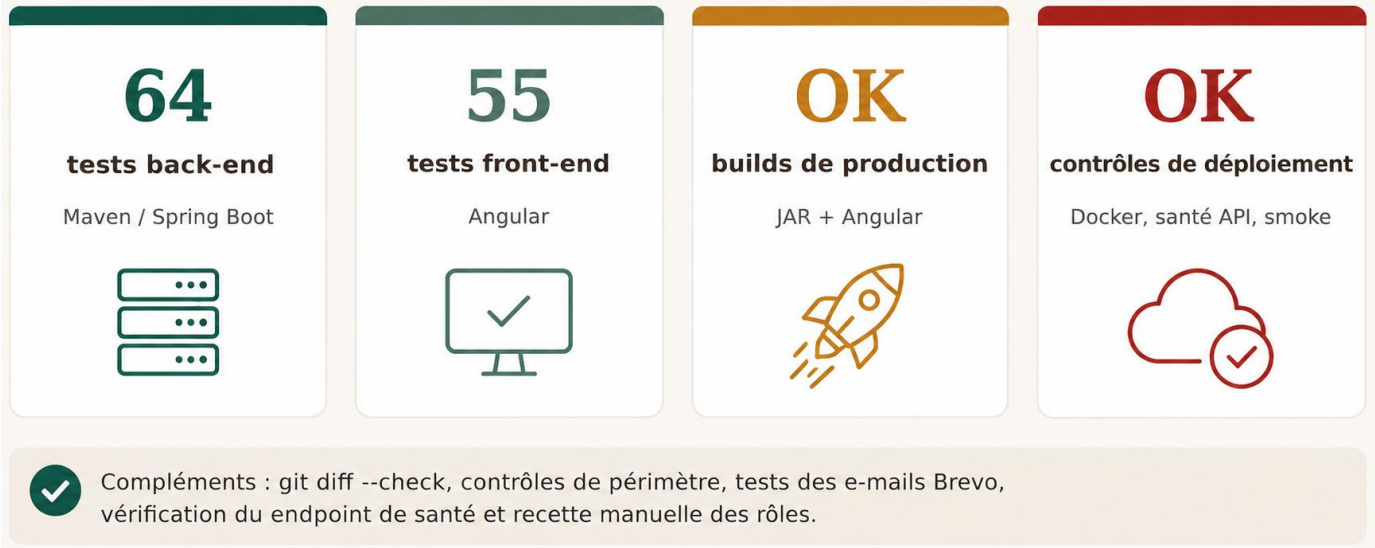


Figure 4 - Contrôles qualité réalisés

5.1 Validation automatisée

Zone	Contrôle	Résultat constaté
Back-end	mvnw clean test	64 tests réussis
Back-end	mvnw clean package -DskipTests	JAR Spring Boot généré
Front-end	Tests Angular	55 tests réussis
Front-end	Build de production Angular	Build réussi
E2E	Playwright et scénarios de pagination	Tests de parcours ciblés
Git	git diff --check et contrôle du périmètre	Aucune erreur d'espacement ni fichier hors périmètre

5.2 Recette fonctionnelle

- Visiteur : accueil, menus, filtres, détail, contact et pages légales.
- Client : inscription, connexion, mot de passe oublié, commande, suivi, profil et avis.
- Employé : consultation, filtrage et traitement des commandes, gestion du catalogue et modération.

- Administrateur : employés, horaires, menus, plats, allergènes et statistiques.
- E-mails : formulaire de contact et réinitialisation vérifiés dans Brevo.

5.3 Validation du déploiement

- Contrôle du endpoint de santé publique de l'API.
- Chargement du front-end avec la configuration runtime de l'URL API.
- Vérification des variables Render sans afficher leur valeur dans le dépôt.
- Contrôle des connexions PostgreSQL Neon, MongoDB Atlas et SMTP Brevo.

6 - Risques, difficultés et actions correctives

Risque ou difficulté	Impact possible	Réponse apportée
Secrets exposés	Accès non autorisé aux services	Variables d'environnement, .gitignore, valeurs factices et rotation d'un mot de passe exposé
Différences local/production	Application fonctionnelle uniquement en local	Profil Spring prod, configuration runtime Angular, Docker et endpoint de santé
CORS et URLs	Requêtes front-end bloquées	FRONTEND_URL et API_URL centralisées sans slash final incohérent
Création du compte admin	Absence d'accès à l'administration	Initialiseur conditionnel puis désactivation après création
Livraison des e-mails	Fonctionnalité non vérifiée	Tests réels via Brevo et contrôle des journaux
Code trop compact	Maintenance difficile	Externalisation Angular et formatage de 78 fichiers Java sans changement fonctionnel
Fichiers non ajoutés à Git	Branche distante vide	Vérification de git status et ajout explicite avant commit
Branche main absente sur le distant	Échec du pull initial	Création de main lors du premier push après fusion de develop
Document absent du projet	paths spec Git introuvable	Copie du PDF dans documentation/ avant git add

6.1 Gestion des risques

- Réduire le périmètre d'une branche pour faciliter le retour arrière.
- Tester avant fusion afin d'éviter de propager une régression.
- Conserver les branches de refactorisation jusqu'à validation du déploiement.
- Utiliser des comptes de démonstration séparés des comptes personnels.
- Prévoir une vérification finale des liens et des documents avant dépôt.

6.2 Retour d'expérience

La principale leçon est que la qualité ne dépend pas uniquement du code. Le suivi Git, la validation du déploiement, la gestion des secrets, les comptes de démonstration et les documents demandés font partie intégrante de la livraison. Les erreurs Git rencontrées ont été corrigées sans perte de travail grâce à une lecture attentive de git status et à des commits réalisés par étape.

7 - Décisions techniques et communication

7.1 Décisions structurantes

Décision	Justification de gestion
Angular standalone pour le front-end	Structure modulaire, composants testables et séparation progressive HTML/SCSS/TypeScript
Spring Boot pour l'API	Organisation claire par contrôleurs, services, entités et repositories
PostgreSQL + MongoDB	Répondre à l'exigence SQL/NoSQL et séparer les données transactionnelles des statistiques
JWT avec refresh token	Sécuriser l'API sans session serveur et gérer les rôles
Docker et Render	Reproduire le démarrage et simplifier le déploiement public
Neon, Atlas et Brevo	Services managés adaptés au contexte d'un projet ECF

7.2 Communication du projet

Le projet étant individuel, la communication ne prend pas la forme de réunions d'équipe. Elle est remplacée par des traces écrites permettant à un tiers de comprendre le travail : README, tickets, noms de branches, messages de commit, documentation technique, manuel utilisateur, charte graphique et copie ECF.

- README : installation locale, configuration et déploiement.
- Issues : suivi des besoins, anomalies et améliorations.
- Commits : historique chronologique des décisions et résultats.
- Documentation : explication des choix, de l'architecture et des parcours.
- Livrables PDF : présentation exploitable par le jury sans environnement technique.

Point de vigilance

Les identifiants présents dans le manuel sont des comptes de démonstration. Les secrets d'infrastructure ne figurent dans aucun document public.

8 - Bilan et axes d'amélioration

8.1 Résultat obtenu

- Application full-stack publique et fonctionnelle.
- Architecture séparant interface, métier, sécurité et données.
- Parcours adaptés aux rôles visiteur, client, employé et administrateur.
- Bases relationnelle et non relationnelle utilisées conformément au sujet.
- Tests automatisés et vérifications de déploiement exécutés.
- Branches main et develop, ainsi que branches fonctionnelles et documentaires.
- Livrables ECF intégrés progressivement au dépôt public.

8.2 Points forts de l'organisation

- Découpage progressif limitant les changements simultanés.
- Traçabilité par les branches, les commits et les fusions explicites.
- Validation technique avant publication dans main.
- Prise en compte tardive mais complète des contraintes de production et de documentation.

8.3 Axes d'amélioration

Amélioration	Application future
Créer le backlog complet dès le démarrage	Saisir tous les tickets avec labels, priorité, critères et estimation
Mettre en place une CI	Automatiser tests et builds à chaque push ou pull request
Utiliser les pull requests systématiquement	Formaliser la revue et conserver les résultats de validation
Ajouter des jalons	Séparer MVP, déploiement, qualité et livraison finale
Renforcer les tests E2E	Automatiser les parcours client, employé et administrateur
Suivre l'accessibilité	Associer des contrôles automatisés et une grille RGAA aux tickets UI

Conclusion. La gestion de projet a permis de transformer un cahier des charges dense en une application livrable. La méthode employée est volontairement pragmatique : elle combine priorisation, gestion des versions, tests et documentation. Les améliorations identifiées constituent une base claire pour conduire un prochain projet de façon encore plus structurée et professionnelle.

A – Annexes opérationnelles

A.1 Commandes Git de référence

Créer une branche

```
git switch develop  
git pull origin develop  
git switch -c feature/nom-de-la-tache
```

Préparer un commit

```
git status  
git diff --check  
git add <fichiers>  
git commit -m "feat: description"
```

Publier la branche

```
git push -u origin feature/nom-de-la-tache
```

Fusionner dans develop

```
git switch develop  
git pull origin develop  
git merge --no-ff feature/nom-de-la-tache  
git push origin develop
```

Publier dans main

```
git switch main  
git pull origin main  
git merge --no-ff develop  
git push origin main
```

A.2 Checklist avant livraison

- ☐ Dépôt GitHub public et branche main à jour.
- ☐ Application frontend et API accessibles.
- ☐ Comptes de démonstration testés.
- ☐ Aucun secret réel présent dans le dépôt.
- ☐ README vérifié.
- ☐ Fichiers SQL de création et d'insertion présents.
- ☐ Manuel utilisateur PDF présent.
- ☐ Charte graphique PDF et six maquettes présentes.
- ☐ Documentation de gestion de projet présente.
- ☐ Documentation technique et diagrammes présents.
- ☐ Copie ECF complétée avec les liens et identifiants.

A.3 Références

Dépôt public : <https://github.com/Scodyx/Vite-et-gourmand>

Gestion de projet : <https://github.com/Scodyx/Vite-et-gourmand/issues>

Documentation GitHub Issues : <https://docs.github.com/fr/issues>

Documentation GitHub - commits : <https://docs.github.com/fr/pull-requests/committing-changes-to-your-project>